

Plano de Integração de Equipa — Cur10usX

Estratégia para transformar o projeto de individual para colaborativo Equipa: 5 pessoas |
Tech Lead: ahamuyel | Data: Maio 2026

Índice

1. [Filosofia de Integração](#)
 2. [Distribuição de Roles](#)
 3. [Onboarding Técnico](#)
 4. [Ordem de Aprendizagem](#)
 5. [Estratégia de Git e Branches](#)
 6. [Estratégia de Code Review](#)
 7. [Ownership por Módulo](#)
 8. [Tarefas por Nível de Experiência](#)
 9. [Cerimónias e Gestão](#)
 10. [Plano de Ação Semanal](#)
-

1. Filosofia de Integração

Princípios

1. **Segurança primeiro** — antes de qualquer feature, garantir que secrets estão seguros e auth é sólida
2. **Documentação como porta de entrada** — ninguém toca em código sem entender a arquitetura
3. **Pequenas vitórias** — primeiras tarefas devem ser pequenas, bem definidas, com entregas rápidas
4. **Code review obrigatório** — zero merges diretos para main
5. **Ownership gradual** — começar com tarefas guiadas, evoluir para ownership de módulos
6. **Testes como requisito** — código novo só é aceite com testes

Abordagem

Semana 1: Leitura + Setup + Tarefas pequenas (supervisionadas)

Semana 2: Features pequenas + Bug fixes (com review)

Semana 3: Módulos completos (ownership partilhado)

Semana 4+: Ownership individual + Features complexas

2. Distribuição de Roles

Estrutura Recomendada (5 pessoas)

Membro	Role Primária	Role Secundária	Foco
M1 — ahamuyel	Tech Lead	DevOps / Architect	Arquitetura, revisão crítica, tarefas complexas, gestão de infra
M2	Frontend Lead	UI/UX Designer	Componentes, landing page, sistema de design, responsividade, acessibilidade
M3	Backend Lead	QA / Testes	API routes, validações, testes unitários/integração, evaluation engine
M4	Fullstack Developer	DevOps Support	CRUD endpoints, formulários, listas, Docker, CI/CD
M5	QA / Documentation	Frontend Support	Testes E2E, documentação, onboarding, bugs, polimento

Responsabilidades Detalhadas

Tech Lead (M1)

- Arquitetura e decisões técnicas
- Code review de todos os PRs
- Gestão de infraestrutura crítica (Docker, K8s, CI/CD)
- Resolução de bugs complexos
- Onboarding e mentoria
- Manter documentação arquitetural
- Garantir padrões de qualidade

Frontend Lead (M2)

- Sistema de design e componentes UI
- Landing page e páginas públicas
- Responsividade e acessibilidade
- Animações e micro-interações
- Consistência visual (dark/light mode)
- Review de PRs de frontend
- Otimização de performance frontend

Backend Lead (M3)

- API routes e lógica de negócio
- Validações Zod e schemas
- Motor de avaliação
- Testes unitários e de integração
- Segurança (auth, CSRF, rate limiting)
- Review de PRs de backend
- Otimização de queries

Fullstack Developer (M4)

- CRUD endpoints e formulários
- Páginas de listagem e detalhe
- Componentes de domínio
- Docker e DevOps support
- Bug fixes intermédios
- Testes de API

QA / Documentation (M5)

- Testes E2E (Playwright/Cypress)
- Testes manuais e regressão
- Documentação de API e guias
- Report e tracking de bugs
- Polimento de UX
- Onboarding docs

3. Onboarding Técnico

Checklist de Onboarding (para cada membro)

Dia 1 — Setup e Contexto (4h)

- ☐ Ler ARCHITECTURE.md (visão geral, stack, arquitetura)
- ☐ Ler AUDITORIA.md (estado atual, issues conhecidas)
- ☐ Configurar ambiente local (npm install, .env, DB)
- ☐ Correr `npx prisma db push && npx prisma db seed`
- ☐ Correr `npm run dev` e verificar app funcional
- ☐ Explorar Prisma Studio (`npx prisma studio`)
- ☐ Navegar pela estrutura de pastas
- ☐ Testar fluxos: login, criar aluno, ver dashboard

Dia 2 — Profundidade por Role (4h)

Frontend:

- ☐ Estudar sistema de componentes (src/components/ui/)
- ☐ Entender providers (auth, theme, school-branding)
- ☐ Analisar 2-3 paginas de listagem completas
- ☐ Analisar 1 formulário CRUD completo
- ☐ Estudar hooks custom (useEntityList, useWebSocket)

Backend:

- ☐ Estudar auth flow (src/lib/auth.ts, middleware.ts)
- ☐ Analisar 2-3 API routes completas
- ☐ Entender validações Zod
- ☐ Estudar evaluation engine
- ☐ Analisar padrão de autorização (requireRole, requirePermission)

Fullstack:

- ☐ Combinar estudos de frontend + backend
- ☐ Mapear fluxo request-API-DB-response-UI

DevOps:

- ☐ Estudar Dockerfile multi-stage
- ☐ Analisar docker-compose e volumes
- ☐ Entender K8s manifests
- ☐ Analisar CI/CD pipeline
- ☐ Configurar deploy local com Docker

Dia 3 — Primeira Contribuição (4h)

- 1. Pegar numa tarefa "iniciante" da secção 8
- 2. Criar branch feature/*
- 3. Implementar com supervisão do Tech Lead
- 4. Submeter PR
- 5. Participar no primeiro code review
- 6. Fazer merge e verificar em produção

4. Ordem de Aprendizagem

Para Todos os Membros

- 1. Conceitos base: Next.js App Router, React Server/Client Components
- 2. Arquitetura geral do projeto (ARCHITECTURE.md)
- 3. Fluxo de autenticação (src/lib/auth.ts, middleware.ts)
- 4. Modelo de dados (Prisma schema)
- 5. Como criar uma API route
- 6. Como criar uma página
- 7. Como criar um componente
- 8. Padrões de validação (Zod)

Frontend First

- 1. Componentes UI existentes (src/components/ui/)
- 2. Hooks custom (src/hooks/)
- 3. Providers (src/provider/)
- 4. Páginas de listagem (src/app/(dashboard)/list/)
- 5. Formulários (src/components/forms/)
- 6. Landing page (src/components/landing/)
- 7. Estilos globais e temas
- 8. i18n (src/lib/i18n/)

Backend First

- 1. API routes existentes (src/app/api/)
- 2. Core libraries (src/lib/)
- 3. Validações Zod (src/lib/validations/)
- 4. Autorização (requireRole, requirePermission)
- 5. Evaluation engine
- 6. Auditoria (audit.ts)

```
7. WebSocket server (ws-server.js)
8. Rate limiting
```

5. Estratégia de Git e Branches

Convenção de Commits (Conventional Commits)

formato: <tipo>(<escopo>): <descrição>

tipos:

feat → Nova funcionalidade

fix → Correção de bug

refactor → Refatoração (sem mudança funcional)

style → UI/UX (css, animações, layout)

test → Adição/modificação de testes

docs → Documentação

chore → DevOps, configs, build, CI/CD

perf → Otimização de performance

security → Correção de segurança

escopos (exemplos):

auth, api, ui, db, ws, docker, k8s, landing, dashboard, forms

exemplos:

feat(api): add student transfer endpoint

fix(auth): validate WebSocket session on connect

style(landing): add scroll reveal animations

test(api): add integration tests for students CRUD

chore(docker): add Redis service to compose

security: rotate exposed secrets

Fluxo de Trabalho Git

```
main                ← produção (protegida, sem pushes diretos)
├── dev              ← integração contínua
│   ├── feat/*       ← novas funcionalidades
│   ├── fix/*        ← correções de bugs
│   ├── refactor/*   ← refatorações
│   ├── style/*      ← UI/UX changes
│   └── test/*       ← testes
```

```
| └─ docs/*           ← documentação
| └─ hotfix/*          ← (da main, para produção urgente)
```

Regras

1. **main** está sempre **deployável** — código testado e revisto
2. **PRs** só para **dev** — exceto hotfixes que vão para **main**
3. **Nunca** fazer **commit** diretamente em **main** ou **dev**
4. **Squash merge** para manter histórico limpo
5. **Branches** curtas — máximo 3 dias de vida
6. **Um PR por tarefa** — se for grande, partir em PRs menores

Estrutura de PR

```
## O que foi feito
[descrição concisa]

# Tipo de mudança
[ ] feat: nova funcionalidade
[ ] fix: correção
[ ] refactor: refatoração
[ ] style: UI/UX
[ ] test: testes
[ ] docs: documentação
[ ] chore: devops

# Como testar
[passos para reproduzir/testar]

# Screenshots (se UI)
[antes/depois]

# Breaking changes?
[sim/não – se sim, descrever]

# Checklist
[ ] Código segue as convenções do projeto
[ ] Testes passam localmente
[ ] Testes novos foram adicionados (se aplicável)
[ ] Documentação foi atualizada (se aplicável)
[ ] PR não expõe secrets ou dados sensíveis
```

6. Estratégia de Code Review

Quem Review O Quê

Tipo de PR	Reviewers
Frontend UI	Frontend Lead + Tech Lead
Backend API	Backend Lead + Tech Lead
Fullstack	Backend Lead + Frontend Lead + Tech Lead
DevOps/Infra	Tech Lead
Testes	Backend Lead ou Tech Lead
Documentação	Tech Lead
Hotfix	Tech Lead (merge rápido)

Critérios de Review

Funcionalidade

- ☐ O código faz o que deveria fazer?
- ☐ Os edge cases estão cobertos?
- ☐ Os estados de loading/error/empty estão implementados?

Código

- ☐ Segue as convenções do projeto?
- ☐ É legível e bem estruturado?
- ☐ Não há duplicação desnecessária?
- ☐ Tipos TypeScript estão corretos?

Segurança

- ☐ Input validation com Zod?
- ☐ Autorização correta (requireRole)?
- ☐ Não há exposição de dados sensíveis?
- ☐ CSRF protegido?

Performance

- ☐ Queries DB estão otimizadas?
- ☐ Evita N+1 queries (Prisma include/select)?

- ☐ Bundle impact considerado?

Testes

- ☐ Testes unitários para nova lógica?
- ☐ Testes de integração para novos endpoints?
- ☐ Testes passam?

SLA de Review

- **PR pequeno** (< 200 linhas): review em 4h
- **PR médio** (200-500 linhas): review em 24h
- **PR grande** (> 500 linhas): review em 48h (recomendar dividir)

7. Ownership por Módulo

Definição de Ownership

Módulo	Primary Owner	Secondary Owner	Área
Autenticação	M1 (Tech Lead)	M3 (Backend)	src/lib/auth.ts, src/app/api/auth/
Gestão Escolar (multi-tenant)	M1 (Tech Lead)	M4 (Fullstack)	School CRUD, applications
Alunos	M3 (Backend)	M4 (Fullstack)	students, enrollments, academic-history
Professores	M4 (Fullstack)	M3 (Backend)	teachers, teacher-subjects, teacher-classes
Turmas	M4 (Fullstack)	M2 (Frontend)	classes, lessons
Disciplinas	M3 (Backend)	M4 (Fullstack)	subjects, courses, global-catalog
Avaliação/Notas	M3 (Backend)	M1 (Tech Lead)	results, evaluation-engine, grading-config
Presenças	M4 (Fullstack)	M3 (Backend)	attendance
Horários	M2 (Frontend)	M4 (Fullstack)	lessons, BigCalendar
Comunicação	M2 (Frontend)	M5 (QA)	messages, announcements, notifications
Suporte	M5 (QA/Docs)	M2 (Frontend)	support-tickets
Dashboard/Stats	M2 (Frontend)	M1 (Tech Lead)	charts, analytics

Módulo	Primary Owner	Secondary Owner	Área
Sistema de Design	M2 (Frontend)	M5 (QA/Docs)	components/ui/, themes
Landing Page	M2 (Frontend)	M5 (QA/Docs)	components/landing/
Importação Dados	M4 (Fullstack)	M3 (Backend)	import, CSV/XLSX
WebSocket	M1 (Tech Lead)	M3 (Backend)	ws-server.js, ws-broadcast
DevOps/Infra	M1 (Tech Lead)	M4 (Fullstack)	Docker, K8s, CI/CD
Testes	M3 (Backend)	M5 (QA/Docs)	todos os testes
Documentação	M5 (QA/Docs)	M1 (Tech Lead)	docs/, README
i18n	M2 (Frontend)	M5 (QA/Docs)	src/lib/i18n/

Matriz de Responsabilidade (RACI)

Atividade	M1 (TL)	M2 (FE)	M3 (BE)	M4 (FS)	M5 (QA)
Decisões Arquiteturais	R	C	C	I	I
Implementação UI	I	R	I	C	C
Implementação API	I	I	R	C	I
Testes Unitários	C	I	R	C	C
Testes E2E	I	I	I	I	R
Code Review	R	C	C	C	I
Documentação	C	C	C	I	R
DevOps/Infra	R	I	I	C	I
Bug Fixes	C	C	C	R	C
Performance	R	C	C	I	I
Acessibilidade	I	R	I	I	C
Segurança	R	I	C	I	I

Legenda: R = Responsible (executa), A = Accountable (responde), C = Consulted, I = Informed

8. Tarefas por Nível de Experiência

Nível 1 — Iniciante (semana 1)

Tarefa	Descrição	Owner Ideal	Estimativa
L1	Adicionar empty states nas listas (mensagem "Nenhum registro encontrado" + CTA)	M2, M5	4h
L2	Adicionar loading skeletons nas páginas de listagem	M2	6h
L3	Criar .env.example na raiz com todas as variáveis documentadas	M4	1h
L4	Remover dependências não usadas do package.json	M4	2h
L5	Escrever testes para schemas Zod (validação de input)	M3, M5	4h
L6	Adicionar SortButton nas colunas da tabela de alunos	M2	2h
L7	Verificar e corrigir contrastes de cor no dark mode	M2	4h
L8	Adicionar meta tags SEO na landing page	M5	2h
L9	Criar CONTRIBUTING.md com guia de contribuição	M5	4h
L10	Adicionar aria-labels em botões de ação (editar, eliminar)	M2	3h

Nível 2 — Intermédio (semanas 2-3)

Tarefa	Descrição	Owner Ideal	Estimativa
M1	Landing page animações de scroll (fade-in/up)	M2	8h
M2	Implementar sistema de toast global para feedback CRUD	M2	6h
M3	Responsividade mobile — tabelas com scroll horizontal	M2, M5	12h
M4	Implementar lazy loading para componentes pesados (charts, calendar)	M2	4h
M5	Adicionar ISR na landing page (revalidate: 60)	M4	2h
M6	Testes unitários — comparePassword, sessionVersion	M3	8h
M7	Testes unitários — evaluation engine (cálculo médias, pesos)	M3	8h
M8	Adicionar serviço Redis ao docker-compose	M4	2h
M9	Migrar rate limiting para Redis	M4, M1	4h
M10	CI: adicionar build step + deploy automático para Vercel	M1, M4	8h
M11	Separar components/ui de domain/ (mover StudentDashboard, etc.)	M2	4h
M12	Adicionar paginação server-side consistente em 5 listas	M4	8h
M13	Adicionar validação Zod client-side nos formulários (useFormState)	M2, M3	8h

Tarefa	Descrição	Owner Ideal	Estimativa
M14	Adicionar filtro "limpar tudo" nos FilterPanels	M2	3h
M15	Criar README multilíngua (EN)	M5	4h

Nível 3 — Avançado (semanas 3+)

Tarefa	Descrição	Owner Ideal	Estimativa
A1	Reforçar WebSocket auth (validar sessão na conexão)	M1, M3	4h
A2	Separar ws-server em serviço independente com Redis pub/sub	M1	8h
A3	Criar helper genérico CRUD para API routes	M3	8h
A4	Criar template genérico de listas (ListPage config-driven)	M2, M4	16h
A5	Extrair services layer das API routes	M3, M1	24h
A6	Testes de integração para auth flow completo	M3	16h
A7	Testes de integração para API CRUD principal	M3	24h
A8	Configurar Playwright/Cypress para E2E	M5	20h
A9	Redis cache para queries de dashboard	M1, M3	8h
A10	Migrar rate limiting para Redis distribuído	M1, M4	4h
A11	Extrair domínio de avaliação para microserviço (fase 2)	M1, M3	40h
A12	Implementar background jobs (Bull + Redis) para importação	M3, M4	12h

9. Cerimónias e Gestão

Cerimónias Scrum Adaptadas

Cerimónia	Frequência	Duração	Participantes	Agenda
Daily Standup	Diária	10-15min	Toda a equipa	O que fiz ontem? O que vou fazer hoje? Há bloqueios?
Sprint Planning	Quinzenal	1-2h	Toda a equipa	Refinar backlog, estimar, definir sprint goal
Code Review Session	2x/semana	30min	Tech Lead + autores	Revisão de PRs pendentes em grupo
Sprint Review	Quinzenal	30min	Toda a equipa	Demo do que foi feito, feedback

Cerimónia	Frequência	Duração	Participantes	Agenda
Retrospective	Quinzenal	30min	Toda a equipa	O que correu bem? O que melhorar? Ações
Tech Sync	Semanal	30min	Tech Lead + Leads	Decisões técnicas, arquitetura, dívida técnica

Ferramentas de Gestão

Ferramenta	Uso
GitHub Projects	Kanban board (To Do, In Progress, Review, Done)
GitHub Issues	Tracking de bugs e tarefas
Discord/Slack	Comunicação diária
Notion/Google Docs	Documentação partilhada
Figma	Design collaboration (se aplicável)

Sprint Template

```

Sprint: #X (DD/MM - DD/MM)
Sprint Goal: [objetivo principal]

Backlog:
[ ] Tarefa 1 (owner, estimativa)
[ ] Tarefa 2 (owner, estimativa)
...

Definição de Done:
- Código implementado e testado
- Testes unitários/integração passados
- Code review aprovado
- Merged em dev
- Documentação atualizada (se aplicável)

```

10. Plano de Ação Semanal

Semana 1 — Fundação (22h trabalho coletivo)

Segunda-feira:

[09:00] Kickoff: apresentação do projeto, roles, expectativas

[10:00] Tech Lead faz walkthrough da arquitetura (2h)

[14:00] Cada membro faz setup local (2h)

Terça-feira:

[09:00] Daily standup

[09:15] Frontend team estuda componentes (M2 + M5)

[09:15] Backend team estuda API routes (M3 + M4)

[14:00] Tech Lead: sessão de autenticação e middleware

Quarta-feira:

[09:00] Daily + Code review workshop

[10:00] Tarefas práticas: cada membro pega 1-2 tarefas Nível 1

- M2: L1 (empty states), L2 (skeletons)

- M3: L5 (Zod tests)

- M4: L3 (.env.example), L4 (cleanup)

- M5: L8 (meta tags), L9 (CONTRIBUTING.md)

Quinta-feira:

[09:00] Daily

[09:15] Continuação tarefas Nível 1

[14:00] Tech Lead: revisão de PRs (code review session)

Sexta-feira:

[09:00] Daily

[10:00] Sprint Review (demo das tarefas da semana)

[10:30] Retrospective

[11:00] Sprint Planning (semana 2)

Semana 2 — Consolidação

- M2: M1 (animações), M2 (toast), M3 (responsividade início)

M3: M6 + M7 (testes auth + evaluation), A1 (WS auth)

M4: M8 + M9 (Redis rate limiting), M12 (pagination)

M5: M3 (responsividade apoio), M15 (README EN), L10 (aria labels)

M1: A2 (WS extração), M10 (CI/CD), code reviews

Semana 3 — Aceleração

- M2: M4 (lazy loading), M11 (componentes split), M13 (client validation)

M3: A3 (CRUD helper), A6 (auth integration tests)

M4: M10 (CI/CD com M1), A9 (Redis cache)

M5: A8 (E2E setup), testes manuais

M1: M10, code reviews, arquitetura

Semana 4+ — Autonomia

- Cada membro assume ownership dos seus módulos

M1 foca em code review, arquitetura, tarefas complexas

Tarefas avançadas (A4, A5, A7, A11, A12) distribuídas

Preparação para apresentação e avaliação

11. Preparação para Avaliação

Apresentação do Projeto

Cada membro deve ser capaz de apresentar:

Área	Quem Apresenta	Conteúdo
Visão Geral	Tech Lead	Propósito, stack, arquitetura, decisões técnicas
Autenticação	Backend Lead	Fluxo de auth, 2FA, OAuth, segurança
Modelo de Dados	Backend Lead	Schema, relações, multi-tenant, evaluation engine
Frontend	Frontend Lead	Componentes, sistema de design, landing page, responsividade
Infraestrutura	Fullstack/DevOps	Docker, K8s, CI/CD, deploy
Testes/Qualidade	QA	Estratégia de testes, cobertura, bugs conhecidos
Módulos Individuais	Cada Membro	Seu módulo: o que faz, como implementou, decisões

Documentação para Avaliação

- `ARCHITECTURE.md` — documentação técnica completa
- `AUDITORIA.md` — análise crítica do projeto
- `BACKLOG.md` — planejamento e tarefas
- `README.md` — visão geral do projeto (já existe)
- `docs/plataforma-por-rolle.md` — funcionalidades por perfil (já existe)

12. Estratégias de Comunicação

Canais

Canal	Uso
Daily Standup	Atualizações rápidas, desbloqueio
GitHub Issues	Tracking de trabalho
GitHub PR Comments	Discussão técnica de código
Discord/Slack Dedicado	Questões rápidas, partilha de recursos

Regras de Comunicação

1. **Issues: fechar no PR** — usar "Closes #123" na descrição do PR
2. **Dúvidas técnicas: GitHub Discussion ou Thread no chat**
3. **Bloqueios: mencionar no daily + ping no chat**
4. **Decisões: documentar no PR ou issue**
5. **Feedback: construtivo e específico, nunca pessoal**

13. Métricas de Progresso

Métrica	Alvo Semana 1	Alvo Semana 4
PRs merged	10-15	5-8/semana
Testes unitários	0 → 15+	50+
Cobertura de testes	0% → 5%	20%+
Tarefas Nível 1 completas	10/10	-
Tarefas Nível 2 completas	0	8/14
Bugs fechados	2-3	10+
Documentação atualizada	ARCHITECTURE + AUDITORIA + BACKLOG	+ CONTRIBUTING + API docs

Nota Final: Este documento é um ponto de partida. A equipa deve adaptá-lo à medida que descobre o que funciona melhor. O objetivo não é seguir rigidamente, mas sim criar uma base de colaboração que permita a todos contribuir eficazmente e crescer como equipa.